بھائی-lang: A Kinship-Typed Programming Language for Cryptographically Verifiable Data Lineage

Adil
Karachi

Abstract. Conventional data-lineage tools (OpenLineage, DataHub, Marquez) treat provenance as an annotation layer over heterogeneous systems and recognise at most three coarse edge types: DERIVED_FROM, DEPENDS_ON, and CONTAINS. We argue this vocabulary is structurally insufficient for the lineage questions modern compliance, ML governance, and AI auditability actually demand.

We present بھائی-lang, a programming language in which data lineage is the runtime substrate rather than a post-hoc instrumentation layer. Every value is a کردار (*kirdaar* — character) carrying its ancestry; every primitive operation extends a typed kinship graph in which ∼50 distinct edges encode relationships such as سوتن (*sautan*: fallback parent), سمدھی (*samdhi*: cross-table-join co-source), استاد (*ustaad*: training-data → model), and گواہ (*gawah*: audit observer). The vocabulary is drawn directly from Urdu kinship terms, for which English-derived data-systems language has no equivalents.

Every کردار is content-addressable via a Merkle-chained SHA-256 we call گواہی (*gawahi* — testimony). The cryptographic structure makes lineage *verifiable* rather than vendor-asserted: a recipient of a serialised lineage graph can confirm, without trusting the producer, that no value or edge has been altered.

We describe the language design, type system, cryptographic protocol, and a ∼1,500-line reference implementation. We illustrate the approach on four case studies: GDPR delete cascade, ML feature PII leak detection, JOIN provenance via a kinship-typed merge, and tamper detection. We argue that lineage-native runtime semantics are technically necessary, vocabulary-rich kinship typing is conceptually overdue, and Merkle-chained provenance is procurable.

1 Introduction

Data-lineage failures are expensive. The European Union has issued more than €4.5 billion in GDPR penalties since 2018, a substantial fraction tied to inadequate ability to identify and delete derived data on request [1]. ML governance teams cannot reliably answer "did sensitive feature X contribute to this prediction?" [2], [3]. AI agents call other agents in nondeterministic chains whose provenance disappears within hours [4].

The dominant industry response is bolt-on instrumentation. Tools such as OpenLineage [5], DataHub [6], and Marquez [7] expose APIs that data-producing systems may call to record lineage events. They are voluntary, eventually consistent, and — critically — operate over a vocabulary of three relationships: DERIVED_FROM, DEPENDS_ON, and CONTAINS. This impoverished type system cannot distinguish

- a value derived from training data versus one derived from inference inputs;
- a join between two source tables versus an inheritance from a single one;
- a fallback computation that consulted a backup source versus a primary path;
- an externally imported value versus a locally derived one;
- a tampered ancestor versus a legitimate one.

Yet these are exactly the distinctions that compliance, ML, and security teams are paid to enforce. The vocabulary gap is not a missing feature; it is structural.

1.1 Insight

Human kinship languages have ∼50 distinct relationship terms because we needed them. Cousin-via-paternal-uncle (چچازاد) is computationally distinct from cousin-via-maternal-aunt (خالہ‌زاد); co-wife (سوتن) is computationally distinct from sister (بہن); witness (گواہ) is computationally distinct from guarantor (ضامن). Karachi Urdu, as a melting-pot dialect, encodes one of the world's richest kinship vocabularies. Mapping this vocabulary onto data lineage yields ex-

actly the relationship distinctions data systems have always lacked names for.

1.2 Contributions

1. A programming-language design (بھائی-lang) in which lineage is the runtime substrate rather than an annotation layer (§3).
2. A kinship-typed lineage edge ontology of ∼50 distinct relationships, with formal computational semantics for the core subset (§4).
3. A cryptographic testimony protocol (گواہی) that makes lineage Merkle-chain verifiable, with $O(N)$ verification cost (§5).
4. A reference implementation in ∼1,500 lines of Python and four case studies (§6, §7).

2 Background and the Vocabulary Gap

2.1 The PROV-DM lineage

The W3C PROV data model [8] formalises provenance as a tripartite graph of *Entities*, *Activities*, and *Agents*, connected by edges including `wasGeneratedBy`, `wasDerivedFrom`, `wasAssociatedWith`, and `actedOnBehalfOf`. PROV-DM is influential and well-specified, but its edge alphabet is small and operates at the granularity of Activities (jobs, runs) rather than individual values. Implementations such as OpenLineage [5] preserve the PROV-DM granularity but reduce the edge vocabulary further, typically to a single OUTPUT_OF relation.

2.2 Capability and information-flow languages

Capability-secure languages (E [9], Pony [10], Monte [11]) carry authority alongside values, ensuring that holding a reference confers the right to use it. Information-flow type systems (FlowCaml [12], Jif [13], LIO [14]) propagate confidentiality labels through computation. Both bodies of work bear on lineage but treat it as a property of *access* or *label*, not of derivation *relationship*. They give us secrecy bits, not kinship structure.

2.3 ML lineage tools

Feast [15], Tecton [16], MLflow [17], and Weights & Biases [18] record feature and experiment lineage. Their semantics are nominal — every artefact has a name and a parent set — but the edges are typed only as PRODUCED or USED. "Did this feature derive from sensitive data?" is, in 2025, still answered by manual code audit at every team we have spoken to.

2.4 Verifiable provenance

Git [19] pioneered the use of a Merkle DAG to make commit history tamper-evident. Certificate Transparency [20] extends the idea to TLS certificate logs. Datalog-with-provenance research [21] expresses lineage as a semiring annotation on relational query results. None of this work has, to our knowledge, been carried into a runtime where individual program values are themselves Merkle-chained.

2.5 The vocabulary gap, quantified

System	Distinct edge types
OpenLineage [5]	3
DataHub [6]	4
Marquez [7]	3
PROV-DM [8]	13
Atlan [22]	5
بھائی-lang (this work)	**∼50**

Table 1: Edge-type expressivity across data-lineage systems. PROV-DM is the formal academic baseline; commercial tools have collapsed it further. بھائی-lang ships with ∼50 typed edges drawn from Urdu kinship vocabulary.

3 Design: Lineage as Runtime Substrate

3.1 The کردار value model

Every value in بھائی is a کردار (kirdaar) — a record carrying:

- a primitive value (number, string, boolean, list, null);
- a set of typed parent edges $\{(r_i, k_i)\}$, each r_i a relation label drawn from the rishta vocabulary, k_i another کردار
- a consent state {VIP, DO-NUMRI, SHOPPING};
- an optional trust label {جانی, بھائی, ⊥};
- an optional sensitivity label {حساس, ⊥};
- an adversarial bit دشمن
- an origin marker {رضاعی, پالک_لے, ⊥}.

3.2 Operations and edge construction

Every primitive operation *constructs* edges. Binary arithmetic `c = a + b` yields a fresh کردار c with parents$(c) = \{(\text{باپ}, a), (\text{ماں}, b)\}$ — left operand as

paternal lineage, right operand as maternal. Higher-arity operations (joins, training declarations) construct correspondingly typed edges.

There is no opt-in. Lineage is created *by construction* as a side effect of computation; it cannot be omitted, forgotten, or buggy.

3.3 Propagation rules

The runtime enforces three propagation laws on every operation, illustrated for the binary case:

$$\text{trust}(c) = \max(\text{trust}(a), \text{trust}(b))$$
$$\text{sens}(c) = \text{sens}(a) \vee \text{sens}(b)$$
$$\text{dushman}(c) = \text{dushman}(a) \vee \text{dushman}(b)$$

Trust is monotone under join, sensitivity and adversarial flags propagate as taint, and consent demotes upon any compromised ancestor.

4 The رشتہ Type System

4.1 Edge vocabulary

The full ontology contains ∼50 entries derived from the kinship structure of Urdu, including paternal-versus-maternal nephew (بھتیجا vs بھانجا), co-brothers-in-law via married sisters (ہم‌زلف), milk-kin (رضاعی), and adopted (لے‌پالک) — each with a distinct semantic. Selected core edges:

Edge	Literal Urdu	Computational semantic
باپ	father	primary derivation (left operand)
ماں	mother	primary derivation (right operand)
استاد	teacher	training-data → model edge
شاگرد	apprentice	inverse of استاد
سوتن	co-wife	fallback parent / `COALESCE`
گواہ	witness	read-only audit snapshot
بیوی / شوہر	wife / husband	JOIN — merged value
سمدھی	co-in-laws	JOIN-induced cross-source rel.
ہم‌زلف	co-brothers-in-law	parallel pipelines, common JOIN
بھتیجا	paternal nephew	paternal-side sibling pipeline
بھانجا	maternal nephew	maternal-side sibling pipeline
چچازاد	paternal cousin	shares paternal grandparent
رضاعی	milk-kin	cached / mirrored copy
لے‌پالک	adopted	externally imported
دشمن	enemy	adversarial mark; do not derive from

Table 2: Core typed lineage edges and their computational semantics. Each edge is a Karachi-Urdu kinship term whose social meaning maps directly onto a data-systems relationship that conventional vocabulary cannot name.

4.2 Why vocabulary matters

Three edges in particular have no name in any data-systems tool we surveyed:

سوتن (co-wife) as fallback parent. A common pattern in resilient computation is: derive a value from a primary source, but fall through to an alternative if the primary is unavailable. SQL implements this as `COALESCE`; ML pipelines implement it as model ensembles or A/B tests. In each case, the *relationship* between the two parents and the child is absent from the lineage

graph. بھائی encodes it as a سوتن edge — declaratively, an alternative parent of equal lineage standing.

سمدھی (co-in-laws) as JOIN-induced relation. When tables `users` and `orders` are joined, the rows of the result are children of both. The relationship between `users` and `orders` themselves — they have shared descendants — is what kinship terms *co-in-laws*. Every JOIN in a data warehouse creates a سمدھی relationship; no lineage tool surfaces it. بھائی does so by construction.

استاد (teacher) as training edge. The training-data-to-model edge is the most-asked-about relationship in AI governance and the least-typed in lineage tools. بھائی represents it directly: a model is a شاگرد of its training data; the `#ur[استاد_چین] <model>` query walks only استاد edges, yielding the complete training provenance.

5 Cryptographic Testimony (گواہی)

5.1 Threat model

We assume:

- the producer of a serialised lineage graph may be adversarial;
- the recipient (auditor, regulator, downstream consumer) wishes to verify the lineage without trusting the producer;
- the recipient has the public hash function but no shared secret;
- integrity, not confidentiality, is the property of interest.

This is the standard threat model for tamper-evident logs.

5.2 Hash construction

For every کردار k we define:

$$\text{gawahi}(k) = H(\text{enc}(k.v, k.\tau, P_k))$$

where H is SHA-256, $k.v$ is the primitive value, $k.\tau$ is the tuple of consent / trust / sensitivity / adversarial / origin tags, and

$$P_k = \text{sort}(\{(r_i, \text{gawahi}(k_i)) : (r_i, k_i) \in \text{parents}(k)\})$$

The encoding `enc` is canonical JSON. Because gawahi is defined recursively over parent hashes, the resulting structure is a Merkle DAG isomorphic to the lineage graph itself.

5.3 Properties

Determinism. Two کردار with identical values, tags, and parent hashes have identical گواہی, regardless of name, allocation order, or session.

Tamper-evidence. Modifying any field of any کردار alters its hash and, by recursion, the hash of every descendant. A single-bit change in a root کردار invalidates every گواہی below it.

Verification cost. Linear in the number of کردار: each is hashed once, each parent edge contributes a constant-size string. For the audit graphs we construct in §7 (10–100 nodes), verification completes in milliseconds.

What it does not catch. A producer can elide کردار entirely from the serialised graph — the hash of the omitted node is simply absent. بھائی addresses elision orthogonally via globals tracking, but a determined producer with no external commitment can still omit. We discuss external anchoring in §8.

5.4 Round-trip verification

The `#ur[سنبھال]` primitive emits, alongside each کردار, its گواہی. The `#ur[اٹھا]` primitive recomputes hashes after reconstructing the graph and aborts with the offending name on mismatch. We have validated this end-to-end on a graph of 5 کردار, including a manual single-byte tamper test.

6 Implementation

The reference implementation is two tree-walking interpreters in ∼1,500 lines of Python:

- `bhai.py` — a general-purpose language with closures, control flow, lists, and a 19-builtin standard library (file IO, time, math, regex). Useful for writing real programs that interact with the operating system.
- `rishta.py` — the lineage-native DSL described in this paper. No control flow; programs are sequences of declarations and queries against the lineage graph.

Both interpreters share the same lexer architecture (Urdu identifier handling, Urdu digit literals ۰–۹, Urdu punctuation ، / ۔). They support modules via `#ur[منگوا]` (cycle-detected import). The runtime is a graph of کردار objects with cached-on-demand hash computation. Persistence emits canonical JSON; load verifies hashes before exposing globals.

The implementation is open-source [23] and runs on Python 3.8+ with no third-party dependencies.

7 Case Studies

7.1 GDPR delete cascade

```
حساس جانی "ali@karachi.pk" = کردار صارف
"کردار پروفائل = صارف + " (کراچی)
"کردار رپورٹ = پروفائل + " — اپریل 2026

تپکا صارف
# ⚠ صارف تپکا — 4 اولاد دو نمبری ہوئی
پھوٹ رپورٹ
# رپورٹ: (ہائیڈ) [دو نمبری] [جانی] [حساس]
```

A single **تپکا** (revoke) call cascades through every value derived from **صارف**, marking each **دو نمبری** (tainted). Reads of tainted **کردار** return a hidden sentinel rather than the underlying value. The compliance team's three-week audit becomes a four-token statement.

7.2 ML feature PII leak detection

```
کردار سی_این_آئی_سی = 4210112345678 حساس
کردار عمر = 25
کردار ماڈل_فیچر = سی_این_آئی_سی + عمر
رساؤ ماڈل_فیچر
# ⚠ حساس رساؤ — 2 حساس جد ملے:
#   ← خود: ماڈل_فیچر [VIP] [حساس]
#   ← باپ: سی_این_آئی_سی [VIP] [حساس]
```

The **رساؤ** (leak) query walks ancestors filtered by sensitivity. Any feature whose lineage includes a **حساس کردار** is flagged. This addresses the canonical question of ML governance — *did sensitive data influence this prediction?* — by lineage walk rather than code review.

7.3 JOIN provenance via **سمدھی**

```
کردار صارفین = 100 حساس
کردار آرڈرز = 50
جوڑ صارفین سے آرڈرز بنا فل_ٹیبل

سمدھی صارفین
#   ↔ 50 = آرڈرز [VIP] (via فل_ٹیبل)
```

The **جوڑ** (join) primitive constructs **بیوی / شوہر** edges from the result to each source. The induced **سمدھی** relationship between sources is queryable on demand. This is the cross-table "what touches what" question every BI team computes ad hoc; here it is graph-intrinsic.

7.4 Tamper detection

We construct a 5-node graph, save it via **سنبھال**, and externally modify the value of one **کردار** in the resulting JSON. The subsequent **اٹھا** aborts with:

```
⚠ گواہی نا کام: 'صارف' میں چھیڑ چھاڑ ہوئی!
expected a6b3b9ea073d…, file says e169a0b214be…
```

The hash chain immediately localises tampering to the responsible **کردار**.

8 Discussion

8.1 What this buys

The combination of (a) lineage-by-construction, (b) kinship-typed edges, and (c) cryptographic testimony eliminates three classes of bug at once: lineage absence, lineage misclassification, and lineage tampering. Compliance audits that today require multi-person quarter-long efforts collapse to graph queries that complete in milliseconds and produce machine-verifiable artefacts.

8.2 Limitations

Performance. The reference interpreter is tree-walking. Realistic workloads will require a bytecode VM and incremental hash maintenance.

Mutation. The current language is value-immutable. A versioned-mutation primitive (**سافٹویئر اپڈیٹ**) is on the roadmap.

Elision. Cryptographic testimony catches modification but not omission. External anchoring (commitment to a public log such as Certificate Transparency or a blockchain) closes this gap; the integration is straightforward but unimplemented.

Adoption. The Urdu-script keyword choice is a cultural commitment. We expect a productionised English-keyword variant will be required for enterprise adoption; the underlying semantics transfer unchanged.

8.3 Cultural framing

The decision to express the type system in Urdu kinship terms is deliberate and not decorative. Karachi Urdu's vocabulary is the most relationship-rich substrate we found; the language design is downstream of the vocabulary, not upstream of it. We conjecture that other natural-language kinship systems (Tamil, Mandarin, Yoruba) would yield comparable insight, and that programming-language design has historically been impoverished by drawing only from English-derived ontologies.

8.4 Future work

Four directions are immediate: (i) external anchoring of **گواہی** to public logs; (ii) inter-organisation lineage-passport format for verifiable hand-off between data producers and consumers; (iii) compile-to-Python em-

bedding so existing codebases can opt in incrementally; (iv) more typed edges from the kinship vocabulary not yet exposed.

9 Related Work

Beyond the work surveyed in §2: PASS [24] pioneered OS-level provenance capture; Burrito [25] captured scientific-workflow provenance; Carmine [26] explored provenance for distributed dataflow. None typed provenance edges beyond a small fixed alphabet, and none made provenance cryptographically verifiable at the value level.

In the cryptographic-log space, Trillian [27] is the production substrate behind Certificate Transparency and presents a general append-only Merkle log. Our گواہی differs in being structured over the lineage DAG itself, not over an opaque event sequence.

In programming-language design, work on effect systems [28], [29] and capability languages most closely parallels ours in treating a meta-property of values (effects, authority) as part of the type system. بھائی extends this tradition to lineage, with the distinguishing feature that the relationship — not just the property — is typed.

10 Conclusion

We have argued that data-lineage tools have been built on an impoverished vocabulary of three to thirteen edge types, and that this vocabulary gap is responsible for a wide class of unanswerable compliance, ML, and AI auditability questions. We presented بھائی-lang, a programming language in which lineage is the runtime substrate, edges are typed from a ∼50-term Karachi-Urdu kinship vocabulary, and every value is content-addressable through a Merkle-chained SHA-256 we call گواہی. The reference implementation is small (∼1,500 lines of Python), demonstrates the approach end-to-end, and is freely available.

The contribution of this paper is not the implementation but the conceptual move: *lineage is not an annotation problem; it is a vocabulary problem solved by making derivation relationships first-class.* We invite the data-systems and programming-language communities to consider what other relationship vocabularies — drawn from the world's languages, not from English alone — might similarly enrich our tools.

Bibliography

[1] "GDPR Enforcement Tracker."

[2] MLflow contributors, "MLflow Tracking and Model Registry." 2024.

[3] W. Pienaar and M. D. Balso, "Feast: Feature Store for Machine Learning." 2024.

[4] Anthropic and others, "Constitutional AI and Agent Auditability: Open Problems." 2024.

[5] OpenLineage Project, "OpenLineage Specification." 2024.

[6] LinkedIn DataHub Project, "DataHub: A Generalized Metadata Search and Discovery Tool." 2024.

[7] Marquez Project, "Marquez: Collect, aggregate, and visualize a data ecosystem's metadata." 2024.

[8] L. Moreau, P. Missier, and others, "PROV-DM: The PROV Data Model," Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[9] M. S. Miller, "Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control," in *PhD Dissertation, Johns Hopkins University*, 2006.

[10] S. Clebsch and others, "Pony: An Actor-Model Capabilities-Secure Language." 2024.

[11] "The Monte Programming Language." 2024.

[12] V. Simonet, "The Flow Caml System: Documentation and User's Manual," in *INRIA Technical Report*, 2003.

[13] A. C. Myers, "JFlow: Practical Mostly-Static Information Flow Control," in *POPL*, 1999, pp. 228–241.

[14] D. Stefan, A. Russo, J. C. Mitchell, and D. Mazières, "Flexible Dynamic Information Flow Control in Haskell," in *Haskell Symposium*, 2011.

[15] Feast Project, "Feast: Open Source Feature Store." 2024.

[16] Tecton Inc., "Tecton: The Feature Platform for Machine Learning." 2024.

[17] MLflow Project, "MLflow: An Open Source Platform for the Machine Learning Lifecycle." 2024.

[18] Weights and Biases, "W&B: The AI Developer Platform." 2024.

[19] L. Torvalds, J. Hamano, and others, "Git: Distributed Version Control System." 2024.

[20] B. Laurie, A. Langley, and E. Käsper, "Certificate Transparency," RFC 6962, 2013. [Online].

Available: https://datatracker.ietf.org/doc/html/rfc6962

[21] T. J. Green, G. Karvounarakis, and V. Tannen, “Provenance Semirings,” in *PODS*, 2007, pp. 31–40.

[22] Atlan Inc., “Atlan: The Active Metadata Platform.” 2024.

[23] Adil, “ب‌های‌ی-lang.” 2026.

[24] K.-K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. Seltzer, “Provenance-Aware Storage Systems,” in *USENIX ATC*, 2006.

[25] P. J. Guo and M. Seltzer, “Burrito: Wrapping Your Lab Notebook in Computational Infrastructure,” in *USENIX TaPP*, 2012.

[26] Y. Wu and others, “Provenance for Distributed Dataflow.” 2018.

[27] Google Trillian Project, “Trillian: A Verifiable Append-Only Log.” 2024.

[28] D. Leijen, “Koka: Programming with Row-Polymorphic Effect Types,” in *MSFP Workshop*, 2014.

[29] S. Lindley, C. McBride, and C. McLaughlin, “Do Be Do Be Do,” in *POPL*, 2017.